

CHAPTER 3 - PHP–MySQL Integration

MySQL is one of the most popular relational database systems being used on the Web today. It is freely available and easy to install, however if you have installed WAMP server it already there on your machine. MySQL database server offers several advantages:

- MySQL is easy to use, yet extremely powerful, fast, secure, and scalable.
- MySQL runs on a wide range of operating systems, including UNIX or Linux, Microsoft Windows, Apple Mac OS X, and others.
- MySQL supports standard SQL (Structured Query Language).
- MySQL is ideal database solution for both small and large applications.
- MySQL is developed, and distributed by Oracle Corporation.
- MySQL includes data security layers that protect sensitive data from intruders.

MySQL database stores data into tables like other relational database. A table is a collection of related data, and it is divided into rows and columns.

Each row in a table represents a data record that are inherently connected to each other such as information related to a particular person, whereas each column represents a specific field such as *id*, *first_name*, *last_name*, *email*, etc. The structure of a simple MySQL table that contains person's general information may look something like this:

id	first_name	last_name	email
1	Peter	Parker	peterparker@mail.com
2	John	Rambo	johnrambo@mail.com
3	Clark	Kent	clarkkent@mail.com
4	John	Carter	johncarter@mail.com
5	Harry	Potter	harrypotter@mail.com

Download MySQL Database: If you don't have a PHP server with a MySQL Database, you can download MySQL for free here: <http://www.mysql.com>

Facts about MySQL Database

One great thing about MySQL is that it can be scaled down to support embedded database applications. Maybe it is because of this many people think that MySQL can only handle small and medium-sized systems.

The truth is that MySQL is the de-facto standard database system for web sites with HUGE volumes of both data and end users (like Friendster, Yahoo, and Google).

Tip: Websites like Facebook, Twitter, Wikipedia uses MySQL for their storage need. So you can easily understand what MySQL is capable of.

Look at <http://www.mysql.com/customers/> for an overview of companies using MySQL.

Choosing an API

PHP offers three different APIs to connect to MySQL. Below we show the APIs provided by the `mysql`, `mysqli`, and `PDO` extensions. Each code snippet creates a connection to a MySQL server running on "example.com" using the username "user" and the password "password". And a query is run to greet the user.

Example #1 Comparing the three MySQL APIs

```
<?php
// mysqli
$mysqli = new mysqli("example.com", "user", "password", "database");
$result = $mysqli->query("SELECT 'Hello, dear MySQL user!' AS _message FROM DUAL");
$row = $result->fetch_assoc();// to Fetch a result row as an associative array
echo htmlentities($row['_message']);// to Convert some characters to HTML entities

// PDO
$pdo = new PDO('mysql:host=example.com;dbname=database', 'user', 'password');
$stmt = $pdo->query("SELECT 'Hello, dear MySQL user!' AS _message FROM DUAL");
$row = $stmt->fetch(PDO::FETCH_ASSOC);
echo htmlentities($row['_message']);

// mysql
$c = mysql_connect("example.com", "user", "password");
mysql_select_db("database");
$result = mysql_query("SELECT 'Hello, dear MySQL user!' AS _message FROM DUAL");
$row = mysql_fetch_assoc($result);
echo htmlentities($row['_message']);
?>
```

Recommended API

It is recommended to use either the [mysqli](#) or [PDO MySQL](#) extensions. It is not recommended to use the old [mysql](#) extension for new development, as it was deprecated in PHP 5.5.0 and was removed in PHP 7. A detailed feature comparison matrix is provided below. The overall performance of all three extensions is considered to be about the same. Although the performance of the extension contributes only a fraction of the total run time of a PHP web request. Often, the impact is as low as 0.1%.

Feature comparison

	ext/mysqli	PDO_MySQL	ext/mysql
PHP version introduced	5.0	5.1	2.0
Included with PHP 5.x	Yes	Yes	Yes
Included with PHP 7.x	Yes	Yes	No
Development status	Active	Active	Maintenance only in 5.x; removed in 7.x
Lifecycle	Active	Active	Deprecated in 5.x; removed in 7.x
Recommended for new projects	Yes	Yes	No
OOP Interface	Yes	Yes	No
Procedural Interface	Yes	No	Yes
API supports non-blocking, asynchronous queries with mysqlnd	Yes	No	No
Persistent Connections	Yes	Yes	Yes
API supports Charsets	Yes	Yes	Yes
API supports server-side Prepared Statements	Yes	Yes	No
API supports client-side Prepared Statements	No	Yes	No
API supports Stored Procedures	Yes	Yes	No
API supports Multiple Statements	Yes	Most	No
API supports Transactions	Yes	Yes	No
Transactions can be controlled with SQL	Yes	Yes	Yes
Supports all MySQL 5.1+ functionality	Yes	Most	No

Ways of Connecting to MySQL through PHP

In order to store or access the data inside a MySQL database, you first need to connect to the MySQL database server. PHP offers two different ways to connect to MySQL server: **MySQLi** (Improved MySQL) and **PDO** (PHP Data Objects) extensions.

While the PDO extension is more portable and supports more than twelve different databases, MySQLi extension as the name suggests supports MySQL database only. MySQLi extension however provides an easier way to connect to, and execute queries on, a MySQL database server. Both PDO and MySQLi offer an object-oriented API, but MySQLi also offers a procedural API which is relatively easy for beginners to understand.

Tip: The PHP's MySQLi extension provides both speed and feature benefits over the PDO extension, so it could be a better choice for MySQL-specific projects.

Connecting to MySQL Database Server

In PHP you can easily do this using the `mysqli_connect()` function. All communication between PHP and the MySQL database server takes place through this connection. Here're the basic syntaxes for connecting to MySQL using MySQLi and PDO extensions:

Syntax: MySQLi, Procedural way

```
$link = mysqli_connect("hostname", "username", "password", "database");
```

Syntax: MySQLi, Object Oriented way

```
$mysqli = new mysqli("hostname", "username", "password", "database");
```

Syntax: PHP Data Objects (PDO) way

```
$pdo = new PDO("mysql:host=hostname;dbname=database", "username", "password");
```

The *hostname* parameter in the above syntax specify the host name (e.g. `localhost`), or IP address of the MySQL server, whereas the *username* and *password* parameters specifies the credentials to access MySQL server, and the *database* parameter, if provided will specify the default MySQL database to be used when performing queries.

The following example shows how to connect to MySQL database server using MySQLi (both *procedural* and *object oriented* way) and PDO extension.

MySQLi, Procedural way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$link = mysqli_connect("localhost", "root", "");

// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}

// Print host information
echo "Connect Successfully. Host info: " . mysqli_get_host_info($link);
?>
```

Note: The default username for MySQL database server is `root` and there is no password. However to prevent your databases from intrusion and unauthorized access you should set password for MySQL accounts.

Tip: Setting the `PDO::ATTR_ERRMODE` attribute to `PDO::ERRMODE_EXCEPTION` tells PDO to throw exceptions whenever a database error occurs.

MySQLi, Object-oriented way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$mysqli = new mysqli("localhost", "root", "", "demo");

// Check connection
if($mysqli === false){
    die("ERROR: Could not connect. " . $mysqli->connect_error);
}
// Print host information
echo "Connect Successfully. Host info: " . $mysqli->host_info;
?>
```

PHP Data Objects (PDO) way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
try{
    $pdo = new PDO("mysql:host=localhost", "root", "");

    // Set the PDO error mode to exception
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // Print host information
    echo "Connect Successfully. Host info: " .
    $pdo->getAttribute(constant("PDO::ATTR_CONNECTION_STATUS"));
} catch(PDOException $e){
    die("ERROR: Could not connect. " . $e->getMessage());
}
?>
```

Closing the MySQL Database Server Connection

The connection to the MySQL database server will be closed automatically as soon as the execution of the script ends. However, if you want to close it earlier you can do this by simply calling the PHP `mysqli_close()` function.

MySQLi, Procedural way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$link = mysqli_connect("localhost", "root", "");

// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}

// Print host information
echo "Connect Successfully. Host info: " . mysqli_get_host_info($link);
```

```
// Close connection
mysqli_close($link);
?>
```

MySQLi, Object-oriented way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$mysqli = new mysqli("localhost", "root", "", "demo");

// Check connection
if($mysqli === false){
    die("ERROR: Could not connect. " . $mysqli->connect_error);
}

// Print host information
echo "Connect Successfully. Host info: " . $mysqli->host_info;

// Close connection
$mysqli->close();
?>
```

PHP Data Objects (PDO) way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
try{
    $pdo = new PDO("mysql:host=localhost;dbname=demo", "root", "");

    // Set the PDO error mode to exception
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // Print host information
    echo "Connect Successfully. Host info: " .
    $pdo->getAttribute(constant("PDO::ATTR_CONNECTION_STATUS"));
} catch(PDOException $e){
    die("ERROR: Could not connect. " . $e->getMessage());
}

// Close connection
unset($pdo);
?>
```

Creating MySQL Database Using PHP

Now you've understood how to open a connection to the MySQL database server. In this tutorial you will learn how to execute SQL query to create a database.

Before saving or accessing the data, we need to create a database first. The [CREATE DATABASE](#) statement is used to create a new database in MySQL.

Let's make a SQL query using the `CREATE DATABASE` statement, after that we will execute this SQL query through passing it to the PHP `mysqli_query()` function to finally create our database. The following example creates a database named *demo*.

MySQLi, Procedural way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
```

```

server with default setting (user 'root' with no password) */
$link = mysqli_connect("localhost", "root", "");

// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
// Attempt create database query execution
$sql = "CREATE DATABASE demo";
if(mysqli_query($link, $sql)){
    echo "Database created successfully";
} else{
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}
// Close connection
mysqli_close($link);
?>

```

MySQLi, Object-oriented way

```

<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$mysqli = new mysqli("localhost", "root", "");

// Check connection
if($mysqli === false){
    die("ERROR: Could not connect. " . $mysqli->connect_error);
}
// Attempt create database query execution
$sql = "CREATE DATABASE demo";
if($mysqli->query($sql) === true){
    echo "Database created successfully";
} else{
    echo "ERROR: Could not able to execute $sql. " . $mysqli->error;
}
// Close connection
$mysqli->close();
?>

```

PHP Data Objects (PDO) way

```

<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
try{
    $pdo = new PDO("mysql:host=localhost;", "root", "");
    // Set the PDO error mode to exception
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch(PDOException $e){
    die("ERROR: Could not connect. " . $e->getMessage());
}
// Attempt create database query execution
try{
    $sql = "CREATE DATABASE demo";
    $pdo->exec($sql);
    echo "Database created successfully";
} catch(PDOException $e){
    die("ERROR: Could not able to execute $sql. " . $e->getMessage());
}
// Close connection
unset($pdo);
?>

```

Creating Tables inside MySQL Database Using PHP

In the previous chapter we've learned how to create a database on MySQL server. Now it's time to create some tables inside the database that will actually hold the data. A table organizes the information into rows and columns.

The SQL [CREATE TABLE](#) statement is used to create a table in database.

Let's make a SQL query using the `CREATE TABLE` statement, after that we will execute this SQL query through passing it to the PHP `mysqli_query()` function to finally create our table.

MySQLi, Procedural way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$link = mysqli_connect("localhost", "root", "", "demo");

// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
// Attempt create table query execution
$sql = "CREATE TABLE persons(
    id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(30) NOT NULL,
    last_name VARCHAR(30) NOT NULL,
    email VARCHAR(70) NOT NULL UNIQUE
)";
if(mysqli_query($link, $sql)){
    echo "Table created successfully.";
} else{
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}
// Close connection
mysqli_close($link);
?>
```

MySQLi, Object-oriented way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$mysqli = new mysqli("localhost", "root", "", "demo");

// Check connection
if($mysqli === false){
    die("ERROR: Could not connect. " . $mysqli->connect_error);
}
// Attempt create table query execution
$sql = "CREATE TABLE persons(
    id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(30) NOT NULL,
    last_name VARCHAR(30) NOT NULL,
    email VARCHAR(70) NOT NULL UNIQUE
)";
if($mysqli->query($sql) === true){
    echo "Table created successfully.";
} else{
    echo "ERROR: Could not able to execute $sql. " . $mysqli->error;
```

```

}
// Close connection
$mysqli->close();
?>

```

PHP Data Objects (PDO) way

```

<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
try{
    $pdo = new PDO("mysql:host=localhost;dbname=demo", "root", "");
    // Set the PDO error mode to exception
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch(PDOException $e){
    die("ERROR: Could not connect. " . $e->getMessage());
}

// Attempt create table query execution
try{
    $sql = "CREATE TABLE persons(
        id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
        first_name VARCHAR(30) NOT NULL,
        last_name VARCHAR(30) NOT NULL,
        email VARCHAR(70) NOT NULL UNIQUE
    )";
    $pdo->exec($sql);
    echo "Table created successfully.";
} catch(PDOException $e){
    die("ERROR: Could not able to execute $sql. " . $e->getMessage());
}
// Close connection
unset($pdo);
?>

```

The PHP code in the above example creates a table named *persons* with four columns *id*, *first_name*, *last_name* and *email* inside the *demo* database.

Notice that each field name is followed by a data type declaration; this declaration specifies what type of data the column can hold, whether integer, string, date, etc.

There are a few additional constraints (also called *modifiers*) that are specified after the column name in the preceding SQL statement, like NOT NULL, PRIMARY KEY, AUTO_INCREMENT, etc. Constraints define rules regarding the values allowed in columns.

Please check out the tutorial on [SQL CREATE TABLE statement](#) for the detailed information about syntax, as well as the data types and constraints available in MySQL database system.

Note: Any number of line breaks may occur within a SQL statement, provided that any line break does not break off keywords, values, expression, etc.

Tip: Setting the `PDO::ATTR_ERRMODE` attribute to `PDO::ERRMODE_EXCEPTION` tells PDO to throw exceptions whenever a database error occurs.

Inserting Data into a MySQL Database Table

Now you've understood how to create database and tables in MySQL. In this tutorial you will learn how to execute SQL query to insert records into a table.

The [INSERT INTO](#) statement is used to insert new rows in a database table.

Let's make a SQL query using the `INSERT INTO` statement with appropriate values, after that we will execute this insert query through passing it to the PHP `mysqli_query()` function to insert data in table. Here's an example, which insert a new row to the *persons* table by specifying values for the *first_name*, *last_name* and *email* fields.

MySQLi, Procedural way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$link = mysqli_connect("localhost", "root", "", "demo");
// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
// Attempt insert query execution
$sql = "INSERT INTO persons (first_name, last_name, email) VALUES ('Peter', 'Parker',
'peterparker@mail.com')";
if(mysqli_query($link, $sql)){
    echo "Records inserted successfully.";
} else{
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}
// Close connection
mysqli_close($link);
?>
```

MySQLi, Object-oriented way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$mysqli = new mysqli("localhost", "root", "", "demo");
// Check connection
if($mysqli === false){
    die("ERROR: Could not connect. " . $mysqli->connect_error);
}
// Attempt insert query execution
$sql = "INSERT INTO persons (first_name, last_name, email) VALUES ('Peter', 'Parker',
'peterparker@mail.com')";
if($mysqli->query($sql) === true){
    echo "Records inserted successfully.";
} else{
    echo "ERROR: Could not able to execute $sql. " . $mysqli->error;
}
// Close connection
$mysqli->close();
?>
```

PHP Data Objects (PDO) way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
try{
    $pdo = new PDO("mysql:host=localhost;dbname=demo", "root", "");
    // Set the PDO error mode to exception
```

```

        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    } catch(PDOException $e){
        die("ERROR: Could not connect. " . $e->getMessage());
    }
    // Attempt insert query execution
    try{
        $sql = "INSERT INTO persons (first_name, last_name, email) VALUES ('Peter',
        'Parker', 'peterparker@mail.com')";
        $pdo->exec($sql);
        echo "Records inserted successfully.";
    } catch(PDOException $e){
        die("ERROR: Could not able to execute $sql. " . $e->getMessage());
    }
    // Close connection
    unset($pdo);
?>

```

If you remember from the [preceding sections](#), the *id* field was marked with the `AUTO_INCREMENT` flag. This modifier tells the MySQL to automatically assign a value to this field if it is left unspecified, by incrementing the previous value by 1.

Inserting Multiple Rows into a Table

You can also insert multiple rows into a table with a single insert query at once. To do this, include multiple lists of column values within the `INSERT INTO` statement, where column values for each row must be enclosed within parentheses and separated by a comma.

Let's insert few more rows into the *persons* table, like this:

MySQLi, Procedural way

```

<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$link = mysqli_connect("localhost", "root", "", "demo");
// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
// Attempt insert query execution
$sql = "INSERT INTO persons (first_name, last_name, email) VALUES
        ('John', 'Rambo', 'johnrambo@mail.com'),
        ('Clark', 'Kent', 'clarkkent@mail.com'),
        ('John', 'Carter', 'johncarter@mail.com'),
        ('Harry', 'Potter', 'harrypotter@mail.com')";
if(mysqli_query($link, $sql)){
    echo "Records added successfully.";
} else{
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}
// Close connection
mysqli_close($link);
?>

```

MySQLi, Object-oriented way

```

<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$mysqli = new mysqli("localhost", "root", "", "demo");

```

```
// Check connection
if($mysqli === false){
    die("ERROR: Could not connect. " . $mysqli->connect_error);
}

// Attempt insert query execution
$sql = "INSERT INTO persons (first_name, last_name, email) VALUES
        ('John', 'Rambo', 'johnrambo@mail.com'),
        ('Clark', 'Kent', 'clarkkent@mail.com'),
        ('John', 'Carter', 'johncarter@mail.com'),
        ('Harry', 'Potter', 'harrypotter@mail.com')";
if($mysqli->query($sql) === true){
    echo "Records inserted successfully.";
} else{
    echo "ERROR: Could not able to execute $sql. " . $mysqli->error;
}
// Close connection
$mysqli->close();
?>
```

PHP Data Objects (PDO) way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
try{
    $pdo = new PDO("mysql:host=localhost;dbname=demo", "root", "");
    // Set the PDO error mode to exception
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch(PDOException $e){
    die("ERROR: Could not connect. " . $e->getMessage());
}

// Attempt insert query execution
try{
    $sql = "INSERT INTO persons (first_name, last_name, email) VALUES
            ('John', 'Rambo', 'johnrambo@mail.com'),
            ('Clark', 'Kent', 'clarkkent@mail.com'),
            ('John', 'Carter', 'johncarter@mail.com'),
            ('Harry', 'Potter', 'harrypotter@mail.com')";
    $pdo->exec($sql);
    echo "Records inserted successfully.";
} catch(PDOException $e){
    die("ERROR: Could not able to execute $sql. " . $e->getMessage());
}

// Close connection
unset($pdo);
?>
```

Now, go to phpMyAdmin (<http://localhost/phpmyadmin/>) and check out the persons table data inside demo database. You will find the value for the id column is assigned automatically by incrementing the value of previous id by 1.

Note: Any number of line breaks may occur within a SQL statement, provided that any line break does not break off keywords, values, expression, etc.

Insert Data into a Database from an HTML Form

In the previous section, we have learned how to insert data into database from a PHP script. Now, we'll see how we can insert data into database obtained from an HTML form. Let's create an HTML form that can be used to insert new records to *persons* table.

Step 1: Creating the HTML Form

Here's a simple HTML form that has three text [<input>](#) fields and a submit button.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Add Record Form</title>
</head>
<body>
<form action="insert.php" method="post">
  <p>
    <label for="firstName">First Name:</label>
    <input type="text" name="first_name" id="firstName">
  </p>
  <p>
    <label for="lastName">Last Name:</label>
    <input type="text" name="last_name" id="lastName">
  </p>
  <p>
    <label for="emailAddress">Email Address:</label>
    <input type="text" name="email" id="emailAddress">
  </p>
  <input type="submit" value="Submit">
</form>
</body>
</html>
```

Step 2: Retrieving and Inserting the Form Data

When a user clicks the submit button of the add record HTML form, in the example above, the form data is sent to 'insert.php' file. The 'insert.php' file connects to the MySQL database server, retrieves forms fields using the PHP `$_REQUEST` variables and finally execute the insert query to add the records. Here is the complete code of our 'insert.php' file:

MySQLi, Procedural way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$link = mysqli_connect("localhost", "root", "", "demo");
// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
// Escape user inputs for security
$first_name = mysqli_real_escape_string($link, $_REQUEST['first_name']);
$last_name = mysqli_real_escape_string($link, $_REQUEST['last_name']);
$email = mysqli_real_escape_string($link, $_REQUEST['email']);

// Attempt insert query execution
$sql = "INSERT INTO persons (first_name, last_name, email) VALUES ('$first_name',
'$last_name', '$email')";
if(mysqli_query($link, $sql)){
    echo "Records added successfully.";
} else{
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}
// Close connection
```

```
mysqli_close($link);
?>
```

MySQLi, Object-oriented way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$mysqli = new mysqli("localhost", "root", "", "demo");

// Check connection
if($mysqli === false){
    die("ERROR: Could not connect. " . $mysqli->connect_error);
}
// Escape user inputs for security
$first_name = $mysqli->real_escape_string($_REQUEST['first_name']);
$last_name = $mysqli->real_escape_string($_REQUEST['last_name']);
$email = $mysqli->real_escape_string($_REQUEST['email']);

// Attempt insert query execution
$sql = "INSERT INTO persons (first_name, last_name, email) VALUES ('$first_name',
'$last_name', '$email')";
if($mysqli->query($sql) === true){
    echo "Records inserted successfully.";
} else{
    echo "ERROR: Could not able to execute $sql. " . $mysqli->error;
}
// Close connection
$mysqli->close();
?>
```

PHP Data Objects (PDO) way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
try{
    $pdo = new PDO("mysql:host=localhost;dbname=demo", "root", "");
    // Set the PDO error mode to exception
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch(PDOException $e){
    die("ERROR: Could not connect. " . $e->getMessage());
}

// Attempt insert query execution
try{
    // Create prepared statement
    $sql = "INSERT INTO persons (first_name, last_name, email) VALUES (:first_name,
:last_name, :email)";
    $stmt = $pdo->prepare($sql);
    // Bind parameters to statement
    $stmt->bindParam(':first_name', $_REQUEST['first_name']);
    $stmt->bindParam(':last_name', $_REQUEST['last_name']);
    $stmt->bindParam(':email', $_REQUEST['email']);

    // Execute the prepared statement
    $stmt->execute();
    echo "Records inserted successfully.";
} catch(PDOException $e){
    die("ERROR: Could not able to execute $sql. " . $e->getMessage());
}
// Close connection
unset($pdo);
?>
```

Note: The `mysqli_real_escape_string()` function escapes special characters in a string and create a legal SQL string to provide security against [SQL injection](#).

This is very basic example of inserting the form data in a MySQL database table. You can extend this example and make it more interactive by adding validations to the user inputs before inserting it to the database tables. Please check out the tutorial on [PHP form validation](#) to learn more about sanitizing and validating user inputs using PHP.

How to Get the ID of Last Inserted Row

In the [PHP MySQL insert](#) chapter you've learnt MySQL automatically generate an unique ID for the `AUTO_INCREMENT` column each time you insert a new record or row into the table. However, there are certain situations when you need that automatically generated ID to insert it into a second table. In these situations you can use the PHP `mysqli_insert_id()` function to retrieve the most recently generated ID, as shown in the upcoming example.

For this example we'll use the same *persons* table that we've created in the [PHP MySQL create tables](#) chapter, which has four columns *id*, *first_name*, *last_name* and *email*, where *id* is the primary key column and marked with `AUTO_INCREMENT` flag.

MySQLi, Procedural way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$link = mysqli_connect("localhost", "root", "", "demo");

// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
// Attempt insert query execution
$sql = "INSERT INTO persons (first_name, last_name, email) VALUES ('Ron', 'Weasley', 'ronweasley@mail.com')";
if(mysqli_query($link, $sql)){
    // Obtain last inserted id
    $last_id = mysqli_insert_id($link);
    echo "Records inserted successfully. Last inserted ID is: " . $last_id;
} else{
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}
// Close connection
mysqli_close($link);
?>
```

Assignment1: Do the above example with the MySQLi, object-oriented way and PDO way

Selecting Data from Database Tables

So far you have learnt how to create database and table as well as inserting data. Now it's time to retrieve data what have inserted in the preceding tutorial. The SQL [SELECT](#) statement is used to select the records from database tables. Its basic syntax is as follows:

```
SELECT column1_name, column2_name, columnN_name FROM table_name;
```

Let's make a SQL query using the `SELECT` statement, after that we will execute this SQL query through passing it to the PHP `mysqli_query()` function to retrieve the table data.

Consider our *persons* database table has the following records:

id	first_name	last_name	email
1	Peter	Parker	peterparker@mail.com
2	John	Rambo	johnrambo@mail.com
3	Clark	Kent	clarkkent@mail.com
4	John	Carter	johncarter@mail.com
5	Harry	Potter	harrypotter@mail.com

The PHP code in the following example selects all the data stored in the *persons* table (using the asterisk character (*) in place of column name selects all the data in the table).

MySQLi, Procedural way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$link = mysqli_connect("localhost", "root", "", "demo");

// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
// Attempt select query execution
$sql = "SELECT * FROM persons";
if($result = mysqli_query($link, $sql)){
    if(mysqli_num_rows($result) > 0){
        echo "<table>";
        echo "<tr>";
            echo "<th>id</th>";
            echo "<th>first_name</th>";
            echo "<th>last_name</th>";
            echo "<th>email</th>";
        echo "</tr>";
        while($row = mysqli_fetch_array($result)){
            echo "<tr>";
                echo "<td>" . $row['id'] . "</td>";
                echo "<td>" . $row['first_name'] . "</td>";
                echo "<td>" . $row['last_name'] . "</td>";
                echo "<td>" . $row['email'] . "</td>";
            echo "</tr>";
        }
        echo "</table>";
        // Free result set
        mysqli_free_result($result);
    } else{
        echo "No records matching your query were found.";
    }
} else{
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}
// Close connection
mysqli_close($link);
?>
```

***Assignment2:** Do the above example with the MySQLi, object-oriented way and PDO way.*

Explanation of Code (Procedural style)

In the example above, the data returned by the `mysqli_query()` function is stored in the `$result` variable. Each time `mysqli_fetch_array()` is invoked, it returns the next row from the result set as an array. The [while loop](#) is used to loop through all the rows in the result set. Finally the value of individual field can be accessed from the row either by passing the field index or field name to the `$row` variable like `$row['id']` or `$row[0]`, `$row['first_name']` or `$row[1]`, `$row['last_name']` or `$row[2]`, and `$row['email']` or `$row[3]`.

If you want to use the [for loop](#) you can obtain the loop counter value or the number of rows returned by the query by passing the `$result` variable to the `mysqli_num_rows()` function. This loop counter value determines how many times the loop should run.

Filtering the Records

The [WHERE](#) clause is used to extract only those records that fulfill a specified condition.

The basic syntax of the `WHERE` clause can be given with:

```
SELECT column_name(s) FROM table_name WHERE column_name operator value
```

Let's make a SQL query using the `WHERE` clause in `SELECT` statement, after that we'll execute this query through passing it to the PHP `mysqli_query()` function to get the filtered data.

Limiting Result Sets

The [LIMIT](#) clause is used to constrain the number of rows returned by the [SELECT](#) statement. This feature is very helpful for optimizing the page loading time as well as to enhance the readability of a website. For example you can divide the large number of records in multiple pages using pagination, where limited number of records will be loaded on every page from the database when a user request for that page by clicking on pagination link.

The basic syntax of the `LIMIT` clause can be given with:

```
SELECT column_name(s) FROM table_name LIMIT row_offset, row_count;
```

The `LIMIT` clause accepts one or two parameters which must be a nonnegative integer:

- When two parameters are specified, the first parameter specifies the offset of the first row to return i.e. the starting point, whereas the second parameter specifies the number of rows to return. The offset of the first row is 0 (not 1).
- Whereas, when only one parameter is given, it specifies the maximum number of rows to return from the beginning of the result set.

For example, to retrieve the first three rows, you can use the following query:

```
SELECT * FROM persons LIMIT 3;
```

To retrieve the rows 2-4 (inclusive) of a result set, you can use the following query:

```
SELECT * FROM persons LIMIT 1, 3;
```

Let's make a SQL query using the `LIMIT` clause in `SELECT` statement, after that we will execute this query through passing it to the PHP `mysqli_query()` function to get the limited number of records.

Ordering the Result Set

The [ORDER BY](#) clause can be used in conjugation with the [SELECT](#) statement to see the data from a table ordered by a specific field. The `ORDER BY` clause lets you define the field name to sort against and the sort direction either ascending or descending.

The basic syntax of this clause can be given with:

```
SELECT column_name(s) FROM table_name ORDER BY column_name(s) ASC|DESC
```

Let's make a SQL query using the `ORDER BY` clause in `SELECT` statement, after that we will execute this query through passing it to the PHP `mysqli_query()` function to get the ordered data.

Tip: By default the `ORDER BY` clause sort the results in ascending order. If you want to sort the records in a descending order, you can use the `DESC` keyword.

Updating Database Table Data

The [UPDATE](#) statement is used to change or modify the existing records in a database table. This statement is typically used in conjugation with the `WHERE` clause to apply the changes to only those records that matches specific criteria.

The basic syntax of the `UPDATE` statement can be given with:

```
UPDATE table_name SET column1=value, column2=value2,... WHERE column_name=some_value
```

Let's make a SQL query using the `UPDATE` statement and `WHERE` clause, after that we will execute this query through passing it to the PHP `mysqli_query()` function to update the tables records. Consider the following *persons* table inside the *demo* database:

```
+-----+-----+-----+-----+
| id | first_name | last_name | email |
+-----+-----+-----+-----+
| 1 | Peter      | Parker    | peterparker@mail.com |
| 2 | John       | Rambo     | johnrambo@mail.com   |
| 3 | Clark      | Kent      | clarkkent@mail.com    |
| 4 | John       | Carter    | johncarter@mail.com   |
| 5 | Harry      | Potter    | harrypotter@mail.com  |
+-----+-----+-----+-----+
```

The PHP code in the following example will update the email address of a person in the persons table whose *id* is equal to 1.

MySQLi, Procedural way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$link = mysqli_connect("localhost", "root", "", "demo");
// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
// Attempt update query execution
$sql = "UPDATE persons SET email='peterparker_new@mail.com' WHERE id=1";
if(mysqli_query($link, $sql)){
    echo "Records were updated successfully.";
} else {
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}
// Close connection
mysqli_close($link);
?>
```

Assignment3: Do the above example with the MySQLi, object-oriented way and PDO way.

After update the persons table will look something like this:

id	first_name	last_name	email
1	Peter	Parker	peterparker_new@mail.com
2	John	Rambo	johnrambo@mail.com
3	Clark	Kent	clarkkent@mail.com
4	John	Carter	johncarter@mail.com
5	Harry	Potter	harrypotter@mail.com

Warning: The `WHERE` clause in the `UPDATE` statement specifies which record or records should be updated. If you omit the `WHERE` clause, all records will be updated.

Deleting Database Table Data

Just as you insert records into tables, you can delete records from a table using the SQL [DELETE](#) statement. It is typically used in conjugation with the `WHERE` clause to delete only those records that matches specific criteria or condition.

The basic syntax of the `DELETE` statement can be given with:

```
DELETE FROM table_name WHERE column_name=some_value
```

Let's make a SQL query using the `DELETE` statement and `WHERE` clause, after that we will execute this query through passing it to the PHP `mysqli_query()` function to delete the tables records. Consider the following *persons* table inside the *demo* database:

id	first_name	last_name	email
----	------------	-----------	-------

id	first_name	last_name	email
1	Peter	Parker	peterparker@mail.com
2	John	Rambo	johnrambo@mail.com
3	Clark	Kent	clarkkent@mail.com
4	John	Carter	johncarter@mail.com
5	Harry	Potter	harrypotter@mail.com

The PHP code in the following example will delete the records of those persons from the *persons* table whose *first_name* is equal to John.

MySQLi, Procedural way

```
<?php
/* Attempt MySQL server connection. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
$link = mysqli_connect("localhost", "root", "", "demo");

// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
// Attempt delete query execution
$sql = "DELETE FROM persons WHERE first_name='John'";
if(mysqli_query($link, $sql)){
    echo "Records were deleted successfully.";
} else{
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}
// Close connection
mysqli_close($link);
?>
```

Assignment4: Do the above example with the MySQLi, object-oriented way and PDO way.

After the deletion the *persons* table will look something like this:

id	first_name	last_name	email
1	Peter	Parker	peterparker@mail.com
3	Clark	Kent	clarkkent@mail.com
5	Harry	Potter	harrypotter@mail.com

As you can see the records has been deleted successfully from the *persons* table.

Warning: The `WHERE` clause in the `DELETE` statement specifies which record or records should be deleted. If you omit the `WHERE` clause, all records will be deleted.